

SAMPLE ASSESSMENT

Android Architecture & Performance Audit

A decision-oriented technical assessment for Android and Jetpack Compose products.



ARCHITECTURE • COMPOSE • COROUTINES • RUNTIME • DATA • BUILD

Prepared by Dharmendra Sharma

Senior Mobile Engineer | Kotlin • Jetpack Compose • KMP

Illustrative sample - findings shown are examples, not claims about a specific client application.

Audit purpose

Turn technical symptoms into evidence, priorities, and a safe remediation plan.

THE QUESTIONS THIS ASSESSMENT ANSWERS

Maintainability

Why are changes becoming slower, riskier, or harder to test?

Reliability

Where can lifecycle, concurrency, state, or integration defects emerge?

Performance

What is actually causing startup delay, jank, memory pressure, or wasted work?

Scalability

Are boundaries and dependencies appropriate for product and team complexity?

ASSESSMENT PRINCIPLE

Measure before changing.

A recommendation should be tied to code evidence, profiling data, build behavior, production symptoms, or a clear maintainability risk - not architecture fashion.

OUTPUT

Executive summary

Prioritized findings

Target architecture

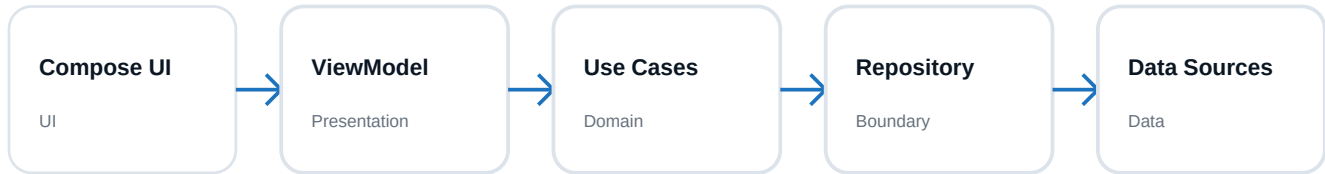
Performance evidence

Remediation roadmap

Architecture & dependency review

Evaluate ownership, dependency direction, state boundaries, modularization and testability.

REFERENCE FLOW



AUDIT CATEGORIES

Boundaries

Business logic placement, framework leakage, ownership.

State

MVVM/MVI/UDF, events vs state, SavedStateHandle.

Modules

Feature/layer boundaries, cycles, shared-core growth.

DI

Scopes, lifecycle ownership, test replacement, Context retention.

Testability

Can core decisions be verified without UI-heavy tests?

Migration risk

Can architecture improve incrementally without a rewrite?

Compose, state & coroutine review

Inspect the boundary where UI state, lifecycle and asynchronous work often create correctness and performance problems.

JETPACK COMPOSE

State hoisting Stability & recomposition remember / rememberSaveable derivedStateOf LaunchedEffect / DisposableEffect Lazy layouts & keys Navigation boundaries

COROUTINES & FLOW

Structured concurrency Cancellation & exception propagation Dispatcher usage StateFlow / SharedFlow combine / flatMapLatest Duplicate collectors Race conditions

DIAGNOSTIC EXAMPLE

Symptom: "This screen is lagging."

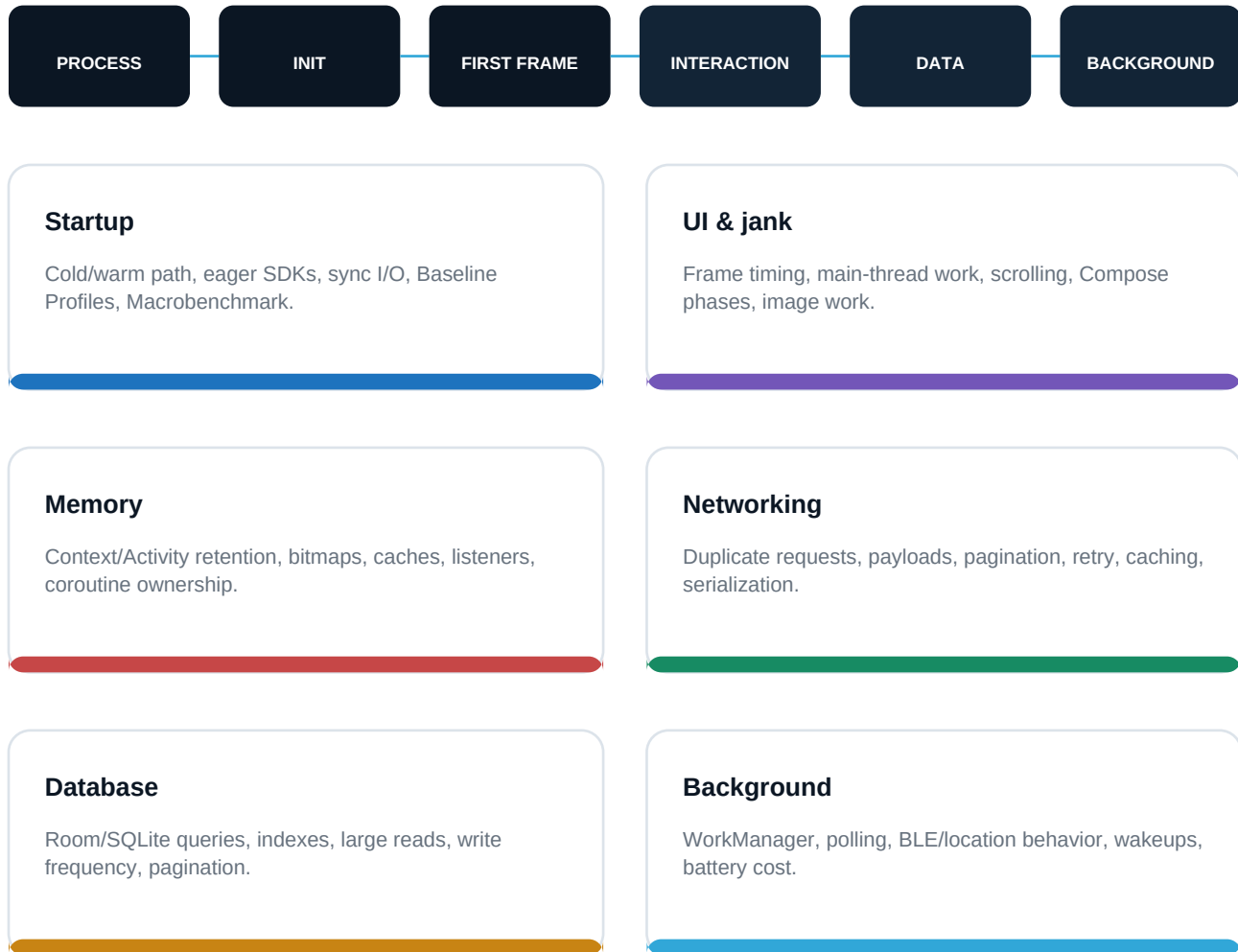
Question: Is the cause recomposition, layout/draw, image decoding, repeated I/O, allocations, or main-thread blocking?

Consulting value = separating the symptom from the measurable cause.

Performance assessment

Profile the application path from process start to user interaction instead of optimizing by intuition.

RUNTIME PATH



Severity scoring & prioritization

Rank findings by production risk and business impact, not by how interesting the refactor looks.

SEVERITY

P0

Production critical

Crash/data/security/release-blocking risk requiring immediate action.

P1

High impact

Material reliability, performance, delivery or maintainability issue.

P2

Important

Meaningful improvement; should enter the planned remediation backlog.

P3

Improvement

Useful cleanup or structural improvement with lower urgency.

PRIORITY MATRIX

IMPACT

P3	P2	P1	P0
P3	P2	P1	P0
P3	P2	P1	P0

URGENCY

Illustrative findings

Example structure: evidence -> impact -> recommendation. These are sample findings only.

P1

Repeated request from UI lifecycle path

- Evidence** Identical API work occurs multiple times during a normal screen visit.
- Impact** Network/battery waste and inconsistent updates.
- Action** Move ownership to ViewModel/use case; make collection lifecycle-aware.

P1

Synchronous work on startup path

- Evidence** Trace indicates non-critical initialization before first frame.
- Impact** Slower cold start and poorer first-use experience.
- Action** Defer non-critical work; benchmark before/after.

P2

Fragmented screen state

- Evidence** Loading/content/error streams can create contradictory combinations.
- Impact** Harder debugging and UI-state defects.
- Action** Model coherent immutable UI state with explicit ownership.

P3

Oversized shared module

- Evidence** Unrelated UI/data/utilities accumulate in one core dependency.
- Impact** Coupling and broader change/build surface.
- Action** Split only across stable responsibility boundaries.

Remediation roadmap

Translate findings into a staged plan that protects delivery while reducing technical risk.

NOW

0-2 weeks

Stabilize

P0/P1 production risks, release blockers, duplicate work, critical lifecycle/concurrency defects.

NEXT

2-6 weeks

Strengthen

State ownership, module boundaries, performance bottlenecks, test seams, build hotspots.

LATER

6+ weeks

Modernize

Broader refactors, KMP/Compose modernization, platform upgrades, long-horizon architecture improvements.

RECOMMENDATION FORMAT

Problem

Evidence

Impact

Action

Effort / Risk

What the client receives

A professional audit should end with a decision package that engineering and product leadership can act on.

01

Executive summary

Top risks, bottlenecks, and recommended direction.

02

Architecture assessment

Boundaries, state, modules, DI, data flow, maintainability.

03

Performance assessment

Startup, UI, memory, networking, database, background work.

04

Prioritized findings

P0-P3 findings with evidence, impact, and recommendation.

05

Target-state diagrams

Practical architecture or flow diagrams for the recommended direction.

06

Remediation roadmap

Now / next / later plan with effort and migration risk.

Typical engagement path

AUDIT -> ENGINEERING SPRINT -> IMPLEMENTATION / MODERNIZATION

Need an independent review of your app?

If your Android product is delayed, unstable, difficult to maintain, or showing performance problems, I can help assess the codebase and define the highest-impact remediation path.

[MESSAGE ME ON LINKEDIN](#)

"AUDIT"

Dharmendra Sharma

Senior Mobile Engineer

Kotlin • Jetpack Compose • KMP • Architecture • Performance

Sample assessment framework. Scope and recommendations are tailored after codebase access and discovery.